Semantic Segmentation on SfM-Reconstructed 3D Point Clouds

Igor Klymenko

April 12, 2026

Abstract

In this work, we propose a method for generating semantically segmented 3D point clouds by combining Structure-from-Motion (SfM) reconstruction with 2D semantic segmentation. Labelled 3D data is essential for spatial reasoning and for training predictive and generative models in 3D space; however, it still remains costly and difficult to acquire and annotate. Most existing approaches rely on manual labelling, LiDAR, or probabilistic methods, which often struggle to scale and generalize due to high acquisition costs and a heavy reliance on geometric shape.

To address this, we combine geometric reconstruction from motion, implemented through a multi-view SfM pipeline, with pre-trained convolutional neural networks (CNNs). Multi-view images are first extracted from the video, and then Lucas–Kanade tracking is applied to the auto-detected features. This tracking is used to obtain an affine reconstruction, which is then upgraded to a metric reconstruction using orthogonality constraints. A pre-trained YOLOv11 model assigns semantic class labels to each pixel in every frame, and these labels are subsequently projected onto the 3D point cloud using majority voting. Finally, a mesh of the reconstructed object is generated via triangulation for evaluation.

The approach is intentionally modular and does not depend on a specific SfM algorithm or segmentation model. Moreover, we introduce metrics to evaluate each module of the pipeline. Our goal is to provide a scalable pipeline for producing accurate, semantically annotated 3D reconstructions without requiring manual labelling or expensive sensors. Our key contribution is a pipeline that combines geometry derived from SfM with per-pixel semantic predictions from CNNs, enabling consistent projection of 2D semantic labels into 3D space.

1 Introduction

Labelled 3D point clouds are important for enabling spatial reasoning, as well as for training both generative models, such as variational autoencoders, and predictive models, such as classification or semantic segmentation models [1]. One practical example is real-time robotic interaction with the environment, where systems need to both navigate space and perform fine-grained actions.

At the same time, current visual-language models are inherently limited by partial observability, as they typically operate on single images and cannot fully capture the underlying 3D geometry of a scene. Because of this, constructing annotated 3D representations becomes important, as it allows models to reason about spatial relationships between objects and opens up new directions for long-term planning in robotics.

With the increasing capability of models to represent complex latent features, more data is required to achieve meaningful results in higher-dimensional function spaces. Despite this demand, the collection of such data remains limited due to the high cost of acquisition with LiDAR and the inherent difficulty of annotating 3D point clouds [1]. Additionally, many existing methods rely heavily on geometric shape while ignoring color and texture [2]. On the other hand, 2D models—although easy to train and annotate due to the wide availability of image data—are fundamentally limited in geometric reasoning because they operate on single views. In other words, while 2D models capture appearance well, meaningful spatial understanding typically requires at least two views to estimate depth reliably.

Therefore, to create a scalable and cost-effective pipeline for reconstructing annotated 3D point clouds, we combine SfM algorithms with pre-trained CNNs [3, 4]. In our approach, we first use feature auto-detection and the Lucas–Kanade tracking algorithm to generate an initial point tracking set. We then apply an SfM pipeline to estimate both 3D point locations and camera poses using SVD-based factorization [3]. In parallel, we run a YOLOv11 segmentation model to assign a semantic class to each pixel in every frame [5]. These predictions are then projected into 3D using majority voting across

views, assigning a class label to each reconstructed point [4]. In this work, we present the theoretical background of the approach as well as experimental results.

2 Motivation

With the increase in computational power, model size, and the ability of AI systems to perform reasoning tasks, the demand for segmented data is growing. In order to train models that operate on 3D point clouds or to perform spatial reasoning, we require datasets where the 3D space is partitioned into semantically meaningful regions. However, unlike text or 2D images, 3D point clouds are significantly harder to collect and annotate. Most existing approaches rely on LiDAR or similar sensors and manual labeling, both of which are expensive and time-consuming [1].

At the same time, learning-based approaches for direct 3D reconstruction (e.g., from RGB-D data) often struggle with accuracy and generalization. This is partly due to the need to encode camera geometry and the limited diversity of available datasets [6]. Methods that rely primarily on geometric structure for segmentation have also been proposed, but these tend to ignore texture and color information, which leads to poor performance in scenes with weak geometric features [2].

In contrast, structure-from-motion (SfM) methods provide reliable 3D reconstruction from multi-view images using geometric constraints [3]. At the same time, CNN-based models perform well on 2D semantic segmentation due to the availability of large labeled datasets and their ability to capture texture and color [5].

Therefore, in this work, we combine these two directions: we apply 2D semantic segmentation across multiple views and transfer these labels onto the 3D point cloud reconstructed via SfM, resulting in a semantically annotated 3D representation.

3 Methodology and Formulation

In this work, we propose an approach that combines semantic factorization SfM with a pre-trained semantic segmentation model (YOLOv11) following a CNN architecture to create a semantically segmented 3D reconstruction from a set of multi-view frames.

3.1 Shi–Tomasi Corner Detection and Lucas–Kanade Tracking

Factorization SfM is an algorithm based on an affine camera model and low-rank factorization in order to approximate the motion and shape matrices from the $2F \times P$ measurement matrix, where P is the number of points and F is the number of frames. This measurement matrix is created as the first step by choosing the number of features using Shi–Tomasi corner detection [7] and using Lucas–Kanade tracking to estimate optical flow by solving a least-squares problem for each selected feature [8].

Shi–Tomasi corner detection selects feature points based on the eigenvalues of the second-moment (structure tensor) matrix:

$$G = \sum_{(x,y)\in W} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix},$$

where I_x and I_y are image gradients and W is a local window around each pixel. A point is considered a good feature if

$$\min(\lambda_1, \lambda_2) > \tau,$$

where λ_1, λ_2 are the eigenvalues of G and τ is a threshold, ensuring strong gradients in two orthogonal directions and therefore stable tracking.

Lucas–Kanade tracking takes an image patch $I(x, y, t)$, where $I(x, y, t)$ corresponds to the intensity of the patch at spatial location (x, y) and time step t. This tracking method is used here for its simplicity and computational efficiency; however, it also poses a fundamental limitation by relying on the brightness constancy assumption, formally:

$$I(x + u, y + v, t + 1) \approx I(x, y, t),$$

where (u, v) is the displacement vector.

Using a first-order Taylor expansion approximation, this assumption becomes

$$I_x u + I_y v + I_t = 0,$$

where I_x, I_y, and I_t are the partial derivatives of the image intensity with respect to x, y, and t. The objective is to find the displacement vector (u, v) by applying this constraint over a local window W for each feature. Together, this generates a least-squares problem:

$$\min_{u,v} \sum_{(x,y)\in W} \left(I_x(x,y)\, u + I_y(x,y)\, v + I_t(x,y)\right)^2.$$

This can be written in matrix form as

$$A \begin{bmatrix} u \\ v \end{bmatrix} = b,$$

where

$$A = \begin{bmatrix} I_x(x_1,y_1) & I_y(x_1,y_1) \\ \vdots & \vdots \\ I_x(x_n,y_n) & I_y(x_n,y_n) \end{bmatrix}, \quad b = - \begin{bmatrix} I_t(x_1,y_1) \\ \vdots \\ I_t(x_n,y_n) \end{bmatrix}.$$

The closed-form solution is obtained via least squares:

$$\begin{bmatrix} u \\ v \end{bmatrix} = (A^\top A)^{-1} A^\top b.$$

Applying Lucas–Kanade estimation to every region detected by Shi–Tomasi corner detection allows us to create the previously mentioned $2F \times P$ measurement matrix, where F is the number of frames and P is the number of tracked points. We then use this matrix for the first step of factorization SfM.

3.2 Factorization Structure-from-Motion

Factorization Structure-from-Motion is a 3D reconstruction algorithm that receives a measurement matrix as input and outputs motion (position of the camera in each frame) and shape (the 3D reconstruction) matrices using the affine (weak-perspective) camera model:

$$\begin{bmatrix} x_{fp} \\ y_{fp} \end{bmatrix} = M_f X_p + t_f,$$

where $X_p \in \mathbb{R}^3$ is a projected 3D point, $M_f \in \mathbb{R}^{2\times 3}$ represents the camera motion matrix for frame f, and $t_f \in \mathbb{R}^2$ is the translation vector.

First, we perform centering to move the translation into the measurement matrix for later factorization. This is achieved by subtracting the centroid of each frame from its coordinates:

$$\tilde{x}_{fp} = x_{fp} - \bar{x}_f, \quad \tilde{y}_{fp} = y_{fp} - \bar{y}_f,$$

where $(\bar{x}_f, \bar{y}_f)$ denotes the centroid of all points in frame f.

Then, the translation term t_f is removed, and the projection becomes

$$\begin{bmatrix} \tilde{x}_{fp} \\ \tilde{y}_{fp} \end{bmatrix} = M_f X_p.$$

After centering, we perform Singular Value Decomposition (SVD) on the measurement matrix W and obtain

$$W = U \Sigma V^\top.$$

Given the affine camera model assumption $\text{rank}(W) \leq 3$, we keep only the top three singular values by magnitude and their corresponding vectors:

$$W \approx U_3 \Sigma_3 V_3^\top.$$

Then, the motion and shape matrices are estimated as:

$$M = U_3 \Sigma_3^{1/2}, \quad S = \Sigma_3^{1/2} V_3^\top.$$

3.3 Metric Upgrade

Note that the reconstruction above is correct up to an affine transformation. In other words, the reconstruction is not unique because for any invertible Q, the following holds:

$$W = MS = (MQ)(Q^{-1}S).$$

To resolve this ambiguity, we impose orthonormality constraints on the rows of the motion matrix M:

$$r_{i1}r_{i1}^{\top} = 1, \quad r_{i2}r_{i2}^{\top} = 1, \quad r_{i1}r_{i2}^{\top} = 0.$$

These constraints are used because we aim to upgrade from an affine camera model to a Euclidean one, where the projection is derived from a rotation matrix R, whose rows are orthonormal. Introducing the symmetric matrix $L = QQ^{\top}$, the constraints can be rewritten as:

$$r_{i1}Lr_{i1}^{\top} = 1, \quad r_{i2}Lr_{i2}^{\top} = 1, \quad r_{i1}Lr_{i2}^{\top} = 0.$$

These equations are then stacked and solved for L using least squares. Using the Cholesky decomposition of L, we recover Q and obtain the metric reconstruction and motion as:

$$M_{\text{metric}} = MQ \in \mathbb{R}^{2F\times 3},$$

$$S_{\text{metric}} = Q^{-1}S \in \mathbb{R}^{3\times P}$$

3.4 Semantic Segmentation

Semantic segmentation is a process of assigning a label to each pixel in a 2D image. The best results in such tasks have been achieved using convolutional neural networks (CNNs) such as YOLOv11 [5]. A CNN is a neural network that learns hierarchical feature representations by applying convolutional filters in order to capture both low-level and high-level features, such as edges for precision and semantic information for understanding object classes.

The architecture follows a structure similar to that of variational autoencoders, where the input is compressed into a latent representation by the encoder and then reconstructed with modifications through upsampling by the decoder - in our case, producing class assignments. These models are typically trained on large datasets of images with per-pixel annotations and use cross-entropy as the loss function.

Figure 1 illustrates a typical CNN-based architecture for semantic segmentation.

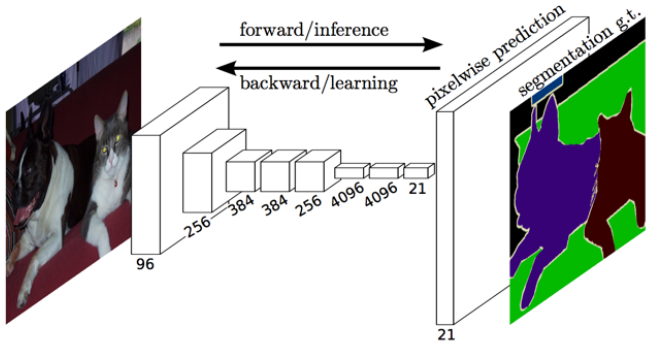


Figure 1: Example of a convolutional neural network architecture for semantic segmentation [9].

In our work, we use a pre-trained YOLOv11 segmentation model to assign semantic labels to each pixel in every frame. The model produces a segmentation mask where each pixel is assigned a class label. We choose YOLOv11 due to the modularity of the pipeline, as it can be easily fine-tuned or replaced with another model. An example of the segmentation is shown in Figure 2, where the model assigns different classes to the table surface, mug, and banana. Due to the scope of this work, we omit the architectural details of YOLOv11 and refer the reader to the original work [5].

3.5 2D Semantic Map to 3D Label Assignment

The last step in the proposed pipeline is the projection step, which projects classes from the 2D segmentation (CNN output) onto the factorization SfM 3D point cloud output. This is achieved through majority voting of classes assigned to the pixels in different frames that correspond to the 3D points.

The output of SfM is the shape matrix $S_{\text{metric}} \in \mathbb{R}^{3\times P}$ from Equation 3.3. Note that each column i represents the 3D position of point i.

YOLOv11 produces a per-frame semantic map seg_maps$[t]$ of the same spatial resolution as the input frame. Each pixel (x, y) is assigned a class label, where 0 corresponds to background. For each tracked point i, its 2D position $(x_{i,t}, y_{i,t})$ is known for each frame t from the tracking stage.

Therefore, despite having class labels for each pixel in the image, we assign labels only to those points that have correspondences in the matrix S. Note that we could use all pixels in the case of dense reconstruction, such as in COLMAP, which leverages multi-view stereo to fill in the space between points in the 3D point cloud [3]; however, our goal here is to obtain a labelled sparse 3D point cloud, and therefore we do not implement dense reconstruction.

Finally, all labels collected across frames that have a correspondence in S are combined into a set, and a majority voting rule is applied to assign a single semantic label to each 3D point:

$$\text{label}_i = \text{mode}\{c_{i,t}\}.$$

This results in one semantic label per reconstructed 3D point. During visualization, each point i is colored according to a predefined palette based on its assigned label.

3.6 Mesh Reconstruction and Texturing

In order to analyze the meaningfulness of the results, we construct a mesh from the reconstructed sparse 3D point cloud. The input consists of the shape matrix S, corresponding image coordinates from the tracking stage, and corresponding labels for each point i.

First, we conduct 3D outlier rejection in order to stabilize rendering. Outliers occur due to errors arising from the tracking and factorization steps. This is achieved through the use of median absolute deviation and radial distance thresholds.

The mesh connectivity is constructed using Delaunay triangulation in 2D image space:

$$\mathcal{T} = \text{Delaunay}(\text{points}_{2D}).$$

This defines triangle connectivity based on the spatial arrangement of points in the image plane, while the corresponding 3D positions from S_{metric} are used as vertex coordinates.

Then, two additional filters are applied to reject triangles with edges in 2D that exceed a threshold relative to the image diagonal and edges in 3D that exceed a threshold relative to the diagonal of the 3D bounding box of the reconstruction.

In addition, for display purposes, we reject triangles that connect two different non-background classes, as they often distort the meshs.

The final mesh is constructed by assigning vertices from S_{metric} and triangle indices from the filtered triangulation. Texture mapping is performed using the corresponding 2D coordinates. For each vertex with image coordinates (x, y), the texture coordinates (u, v) are computed as:

$$u = \frac{x}{w}, \quad v = 1 - \frac{y}{h},$$

where w and h are the width and height of the image, respectively. This allows the original image to be used as a texture for the reconstructed surface.

Finally, each triangle is assigned a color based on the semantic classes obtained in the projection step. Specifically, the triangle color is determined either by directly using the colors of its vertices or by assigning the majority class among its vertices.

4 Experiments

In this section, we describe the results of the proposed approach. First, we present a qualitative reconstruction analysis on a video containing a set of basic objects such as a mug and a banana in a

standard setting, focusing on the results of each step of the pipeline. Then, we evaluate reconstruction and semantic labelling consistency by reporting measurements of the SfM reconstruction and the agreement of the semantic model across different views of the same scene used for class assignment.

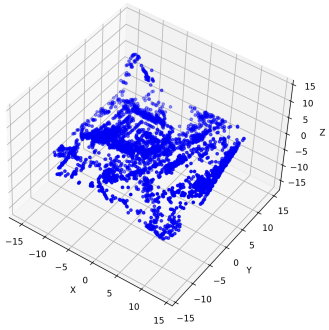
Note that in our implementation we make two crucial assumptions: brightness constancy for Lucas–Kanade tracking and an affine (weak-perspective) camera model for reconstruction. Later, we discuss how these limitations could be addressed.

4.1 Qualitative Reconstruction Analysis

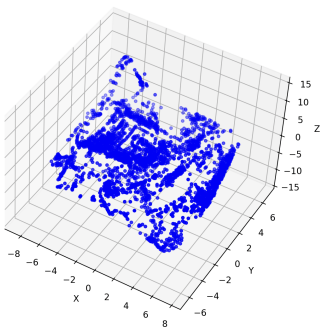
In the figure below, we present each step of the running pipeline with the goal of creating a semantic 3D reconstruction in a standard setting, with two objects of interest being geometrically close to each other. The input to the pipeline is a video captured using a consumer-grade smartphone camera on an iPhone 11. Additional mesh results are provided in Appendix A.



(a) Input frame



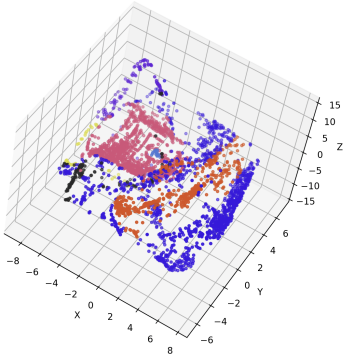
(b) Affine reconstruction



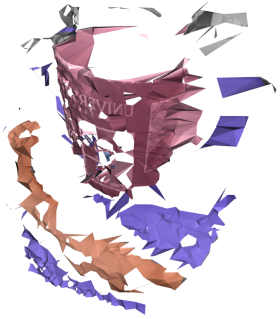
(c) Metric reconstruction



(d) 2D segmentation



(e) Labelled 3D points



(f) Textured mesh

Figure 2: Qualitative results of each step in the proposed pipeline: (a) input frame, (b) affine reconstruction, (c) metric reconstruction, (d) 2D semantic segmentation (classes include cup, banana, and dining table), (e) labelled 3D points, and (f) textured mesh. Each color corresponds to the same class label: pink corresponds to cup, orange to banana, and purple to the table surface.

In Figure 2, each step represents the stages of the proposed pipeline. Figure 2(a) shows the first frame from the 90 frames used for reconstruction. For tracking, we set an upper bound of 3000 features, resulting in 2840 features selected by the Shi–Tomasi corner detection algorithm. These features are reconstructed in affine and then metric space, as shown in Figures 2(b) and 2(c). The change after the upgrade is small; however, it is necessary to eliminate ambiguity in the affine reconstruction.

Figure 2(d) shows semantic segmentation of the first frame, assigning three classes: cup, banana, and table (YOLOv11 class "dining table"). Figure 2(e) shows the result of combining metric reconstruction with 2D segmentation in a segmented 3D point cloud. Figure 2(f) shows the corresponding mesh and its 2D correspondence.

Note that in the point cloud we can see additional colors beyond the three listed. This is due to background labeling and detection of other objects, such as the book visible in Figure 2(a). These objects are not of primary interest and serve as noise in the scene.

In terms of quality, the reconstruction appears stable, with most points concentrated around the objects of interest. This shows the effectiveness of the Shi–Tomasi algorithm in this setting and indicates that the tracking and reconstruction pipeline works stably. However, a line of outliers is still visible above the main structure. This likely corresponds to the background book, which is supported by the light purple label in Figure 2(e). This violates the weak-perspective assumption due to its larger depth, resulting in degenerate behavior.

At the end of the pipeline, Figures 2(e) and 2(f) show promising results, clearly separating objects in both space and class (table, mug, banana). Moreover, the pipeline captures their shapes and relative positions.

4.2 Reconstruction Consistency and Error Analysis

In order to describe the key metrics of the proposed pipeline and understand what contributes most to the error, we consider the experiment described above. In this section, we discuss tracking, SfM, mesh, and segmentation quality through the metrics presented in Table 1.

Table 1: Quantitative evaluation of geometry, meshing, and semantic consistency.

Metric	**Value**
Geometry	
Reprojection error (mean, px)	45.61
Reprojection error (median, px)	23.27
Reprojection error (std, px)	68.83
Mean track length (frames)	76.33
Median track length (frames)	90
Outlier points removed (%)	10.95
Mesh	
Triangles (initial)	5031
Triangles after filtering	3386
Triangles removed (%)	32.7
Cross-class triangle conflicts (%)	8.39
Semantics	
Non-background labelled points (%)	95.53
Mean label consistency	0.91
Median label consistency	1.00
Label entropy (mean)	0.13
Combined	
Consistency $\times$ inverse reprojection	0.020

In this setup, 2840 points were selected and tracked across 90 frames. The mean track length is approximately 76 frames, and the median is 90, indicating that tracking was stable. The SfM reprojection error has a mean of 45 pixels and a median of 23 pixels, with a high standard deviation. These metrics indicate instability in the geometric reconstruction of the proposed pipeline. A major factor in this setting is the violation of the weak-perspective camera assumption, as many tracked points appear at significantly different depths.

From the mesh-related metrics, we observe that the filters removed approximately 33% of the initially generated triangles, which is clearly correlated with the reprojection error from SfM. This also highlights the importance of the filtering steps introduced to achieve more stable visualization.

Finally, the semantic segmentation model performs well, showing high consistency in assigning the same class to the same points, achieving a median consistency of 1.0. Label entropy was measured from the model's internal logits, indicating that the pre-trained YOLO model can confidently recognize standard objects in this setting. Since no ground truth labels are available, we cannot evaluate cross-entropy of predicted classes in either 2D or 3D. Instead, we rely on the results reported in the original work; therefore, the key indicator here is the low entropy produced by the model.

The combined metric is defined as

$$\text{Score} = \text{mean label consistency} \cdot \frac{1}{\text{mean reprojection error}},$$

where a higher score indicates better overall reconstruction quality. In this case, despite visually appealing reconstruction, the score remains low, primarily due to the high reprojection error.

We believe this is due to the choice of the SfM algorithm, which relies on the weak-perspective camera model. Therefore, to improve the results, we suggest implementing bundle adjustment–based SfM to potentially reduce the reprojection error and increase the overall score.

The positive side is that, despite the high reprojection error, the overall results still provide the desired outcome, depicting labelled 3D point clouds that clearly follow the shapes of the objects in the video and are correctly labelled in 3D. Moreover, we believe that, given these results, it may still be possible to perform spatial reasoning or use this data for training predictive models on the key objects of interest by truncating the outliers.

5 Conclusion & Future Work

In this work, we show that SfM algorithms, specifically factorization SfM, can be combined with CNNs to generate a labelled 3D point cloud from a video captured on a consumer-grade phone. We achieve this by providing the relevant theory, including feature auto-selection, tracking, affine reconstruction, upgrade to metric reconstruction, semantic segmentation, 2D to 3D label projection, and meshing. We also present experimental results in a standard setting, where the aim is to reconstruct and segment two different objects in the foreground. We show that, despite high SfM reprojection error, we still obtain a labelled 3D point cloud that allows potential reasoning about the scene through visual cues derived from reconstructed objects and their assigned labels. We also present the reconstructed mesh of the scene and provide metrics to evaluate the performance of each modular component of the pipeline.

Potential future directions include studying different models and algorithms within the pipeline. For example, substituting factorization SfM with bundle adjustment SfM could reduce reprojection error and potentially improve overall results. Another direction is to compare different methods of 3D point cloud labelling against the proposed pipeline using metrics such as the overall score defined above, runtime, and computational resources.

An additional advantage of the proposed approach is that it is relatively inexpensive to run-the above experiment was conducted on a 2020 MacBook Air with an M1 chip. Therefore, given the trade-off between computational cost and accuracy, this can be further evaluated in real applications by providing both high- and low-score outputs to AI agents or by training ML models and measuring performance degradation. This would help estimate the necessity of more computationally expensive SfM algorithms and larger segmentation models.

A Additional Mesh Results



(a) Mesh (view 1) (b) Mesh (view 2) (c) Different scene

Figure 3: Additional mesh reconstruction results. (a) and (b) show different views of the same reconstructed scene, while (c) presents a result on a different scene.

References

[1] Y. He, H. Yu, X. Liu, Z. Yang, W. Sun, S. Anwar, and A. Mian, "Deep learning based 3d segmentation: A survey," *arXiv preprint arXiv:2103.05423*, 2021.

[2] I. Badami, M. Tom, M. Mathias, and B. Leibe, "3d semantic segmentation of modular furniture using rjmcmc," in *Proceedings of the IEEE Winter Conference on Applications of Computer Vision (WACV)*, pp. 64–72, 2017.

[3] J. L. Schönberger and J.-M. Frahm, "Structure-from-motion revisited," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[4] F. R. Capua and P. De Cristoforis, "Mapping semantic segmentation to point clouds using structure from motion for forest analysis," *arXiv preprint arXiv:2505.10751*, 2025.

[5] R. Khanam and M. Hussain, "Yolov11: An overview of the key architectural enhancements," *arXiv preprint arXiv:2410.17725*, 2024.

[6] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: Representing scenes as neural radiance fields for view synthesis," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2020.

[7] J. Shi and C. Tomasi, "Good features to track," in *Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 593–600, 1994.

[8] B. D. Lucas and T. Kanade, "An iterative image registration technique with an application to stereo vision," in *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 674–679, 1981.

[9] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," 2015.